Trabajo Práctico · Unidad 4

Forma Normal de Boyce-Codd y desnormalización controlada en Food Store · Base de Datos II · Tecnicatura Universitaria en Programación · UTN

Alumno: Moran, Joaquín Leandro · Trabajo individual.
Repositorio: github.com/moranjoaquin487-lgtm/tpi-food-store, carpeta `trabajos-practicos/unidad4_tp_semana8`.
Base: copia de prueba `foodstore_tp4` en PostgreSQL 18 (mi PC), armada desde cero con los scripts del proyecto: 200 005 pedidos y 399 994 líneas de los últimos dos años. No se tocó `foodstore`. Todo se corre con `bash trabajos-practicos/unidad4_tp_semana8/ejecutar_tp_u4.sh`.

Entregable	Archivo en el repositorio	Salida de mi corrida
1. Parte 1 (FNBC)	`sql/tp_fnbc_control_lote.sql` (anexo A)	`evidencias/parte1_fnbc_control_lote.txt`
2. Parte 2 (desnormalización)	`sql/tp_desnormalizacion_top_categorias.sql` (anexo B)	`evidencias/parte2_desnormalizacion_top_categorias.txt`
3. Informe	`docs/Informe_TP_U4_Moran.pdf` (este archivo)	—

Parte 1 · `control_lote_almacen` hasta FNBC

Abreviaturas: **L** = LoteID (`lote_id`), **D** = DepositoID (`deposito_id`), **R** = ResponsableControlID (`responsable_control_id`). U = {L, D, R}. Las maestras `lote` y `deposito`, que la consigna da por existentes, se crean con lo mínimo; los responsables 801, 802 y 803 son usuarios de prueba con datos inventados.

a) Dependencias funcionales

DF	Regla de negocio	En la instancia
DF1: {LoteID, DepositoID} → ResponsableControlID	Para un lote y un depósito, el responsable es uno solo.	0 violaciones
DF2: ResponsableControlID → DepositoID	Cada responsable pertenece a un único depósito.	0 violaciones

F = {DF1, DF2}. No se postula L → D ni L → R: un lote grande se fracciona y se controla desde más de un depósito, con un responsable por depósito.

b) Clausuras, claves candidatas y atributos primos

X	X^+ con F	¿Superclave?
{L}	{L}	No
{D}	{D}	No
{R}	{R, D} (DF2)	No
{D, R}	{D, R}	No
{L, D}	{L, D, R} (DF1) = U	**Sí, clave**
{L, R}	{L, R, D} (DF2) = U	**Sí, clave**

L no aparece del lado derecho de ninguna DF: toda clave tiene que contenerlo, y {L} solo no alcanza. Con dos atributos, {L, D} y {L, R} llegan a U y son mínimas, porque ninguno de sus subconjuntos propios es superclave. {L, D, R} es superclave, pero no es mínima.

Claves candidatas: {L, D} (la PK) y {L, R}. **Atributos primos:** L, D y R, los tres. **No primos:** ninguno.

En la instancia, {L, R} no se repite (0 repetidos), y R = 801 aparece en dos filas, así que R solo no identifica una fila.

c) ¿Cumple FNBC?

FNBC exige que, para toda DF no trivial X → Y, X sea superclave. DF1 cumple: {L, D} es clave. **DF2 viola FNBC:** R → D no es trivial (D ∉ {R}) y su determinante no es superclave, porque $\{R\}^+ = \{R, D\} \neq U$: R no determina L. Por eso 801 aparece dos veces, en los lotes 501 y 502, y el dato «801 es del depósito 30» se guarda dos veces. El esquema sí está en 3FN, porque D es primo, pero la definición de FNBC no tiene esa excepción.

d) Anomalías, sobre la instancia de ejemplo

Actualización. El responsable 801 pasa al depósito 32. Como el dato está repetido en las filas de los lotes 501 y 502, hay que cambiar las dos. Si se cambia solo la del lote 501 (`UPDATE 1`), la tabla dice que 801 pertenece a los depósitos 30 y 32 a la vez, y queda rota la regla de que cada responsable es de un único depósito.

Inserción. Se incorpora el responsable 803 al depósito 31, pero todavía no controló ningún lote. No hay forma de registrarlo: `lote_id` es parte de la clave primaria y no admite nulos, y el motor rechaza la fila (NULL, 31, 803) con un error de not-null. El dato maestro «803 es del depósito 31» depende de que exista un control.

Borrado. Se anula el control del lote 503. Era la única fila de 802, así que al borrarla se pierde también que 802 pertenece al depósito 31: después del `DELETE`, ninguna fila dice cuál es su depósito.

Las tres se ejecutan dentro de `BEGIN … ROLLBACK` en el script. La de inserción es uno de los dos errores esperados de la corrida.

e) Descomposición sin pérdida

Algoritmo visto en clase, con la DF violatoria X → Y = R → D: R1 = X ∪ Y = (R, D) y R2 = U − (Y − X) = (L, R). En R1 la única DF es R → D, con R como clave. En R2 no quedan DF no triviales. Las dos están en FNBC.

```
CREATE TABLE responsable_deposito (              -- R1(R, D)
    responsable_control_id BIGINT PRIMARY KEY REFERENCES usuario(id),
    deposito_id            BIGINT NOT NULL    REFERENCES deposito(id));
CREATE TABLE control_lote (                      -- R2(L, R)
    lote_id                BIGINT NOT NULL REFERENCES lote(id),
    responsable_control_id BIGINT NOT NULL REFERENCES responsable_deposito(responsable_control_id),
    PRIMARY KEY (lote_id, responsable_control_id));
CREATE VIEW v_control_lote_almacen AS            -- vista de compatibilidad
SELECT lote_id, deposito_id, responsable_control_id
  FROM control_lote NATURAL JOIN responsable_deposito;
```

DF1 no se preserva: ninguna de las dos tablas tiene L y D juntos. Para no perder la regla, el trigger `fn_control_lote_df1` (por sentencia, sobre `control_lote` y sobre los cambios de depósito en `responsable_deposito`) busca en la vista algún (L, D) repetido y rechaza la operación. Toma un candado de transacción para que dos transacciones concurrentes no pasen el control a la vez. En mi corrida aceptó que 802, del depósito 31, controle también el lote 501, que se controla desde el 30, y rechazó un segundo responsable del depósito 31 para el lote 503 (el otro error esperado).

f) Por qué la reunión es sin pérdida, y migración verificada

Teorema de Heath: la descomposición de U en R1 y R2 es sin pérdida si el atributo común determina todos los atributos de una de las dos. Acá R1 ∩ R2 = {R}, y R es clave de R1: R → D, es decir, R1 ∩ R2 → R1. Cada fila de `control_lote` encuentra, por R, exactamente una fila de `responsable_deposito`, así que la reunión natural no pierde tuplas ni agrega espurias. Además, la FK de `control_lote` hacia `responsable_deposito` garantiza que esa fila exista siempre.

La migración va en una sola transacción: copia los pares (R, D) sin repetir a R1 (2 filas) y los pares (L, R) a R2 (3 filas). Antes del `COMMIT`, un bloque `DO` compara la tabla original con la vista usando `EXCEPT` en los dos sentidos y cancela todo si hay diferencias.

Verificación con la vista de compatibilidad	Resultado
Tuplas perdidas (original − vista)	**0**
Tuplas espurias (vista − original)	**0**
Filas que devuelve la vista	3: (501, 30, 801), (502, 30, 801), (503, 31, 802)

Anomalía	Esquema original	Esquema descompuesto
Actualización	Una fila por lote: 801 queda en {30, 32}	Una sola fila en R1 (`UPDATE 1`)
Inserción	Error: falta `lote_id`	803 se registra en el depósito 31 sin lotes
Borrado	Se pierde el depósito de 802	El depósito de 802 sigue en R1

Parte 2 · Top 5 de categorías del día

Adaptación al esquema real. La consulta de la consigna usa nombres genéricos. En Food Store no hay `dp.subtotal` (el monto es `cantidad * precio_unitario`), las claves se llaman `id_producto`, `id_categoria` e `id_pedido`, y ni `pedido` ni `detalle_pedido` tienen borrado lógico: lo que no cuenta como venta es un pedido `CANCELADO` (columna `estado`, de la U7). `pedido.fecha` es `TIMESTAMPTZ`, así que «el día» es la fecha en la hora de Argentina: `dia_venta(ped.fecha) = hoy()`.

a) La consulta original con EXPLAIN ANALYZE

Tiempo de ejecución: **164,528 ms**. El nodo que domina el costo es el **Seq Scan on pedido**: termina a los 161,429 ms, el 98 % del total. Recorre los 200 005 pedidos y descarta 199 790 para quedarse con los 215 de hoy. El resto del plan son *nested loops* con índices sobre `detalle_pedido` y `producto`, que suman unos pocos milisegundos. El costo crece con la tabla y se paga en cada refresco del panel. Captura completa del plan en la sección «Planes».

b) Patrón elegido y justificación

Elegí una **tabla resumen precalculada mantenida con disparadores** (`ventas_dia_categoria`: una fila por día y categoría, con unidades y monto): es el patrón de columna precalculada con disparador, aplicado a un agregado. **La evidencia** es la de (a): el 98 % de los 164,528 ms se va en leer todos los pedidos para quedarse con 215, y el panel repite esa lectura muchas veces por minuto, cada vez sobre una tabla más grande. Descarté la vista materializada porque el panel pide datos en tiempo real: entre un `REFRESH` y otro mostraría ventas viejas, y cada `REFRESH` vuelve a calcular dos años de ventas, que es justo el costo que quiero evitar. **No se desincroniza** porque los disparadores ajustan el resumen en la misma transacción que modifica la fuente de verdad: altas, cambios y bajas en `detalle_pedido`, cambios de fecha o de estado y borrados en `pedido`, y cambios de categoría en `producto`. Si el ajuste falla, la venta tampoco se guarda. El script de auditoría de (e) lo controla desde afuera. **Es reversible sin pérdida**: la fuente de verdad (`pedido`, `detalle_pedido`) nunca se modifica, así que quitar los triggers, las funciones y la tabla no borra ningún dato; el script lo ensaya con `ROLLBACK` y la consulta original devuelve el mismo top 5. El costo es que cada venta ejecuta un disparador más.

c) Implementación (script completo en el anexo B)

Disparador	Evento	Qué hace en `ventas_dia_categoria`
`trg_ventas_dia_detalle`	`detalle_pedido`: INSERT, UPDATE, DELETE	Resta la línea vieja y suma la nueva, salvo que el pedido esté cancelado.
`trg_ventas_dia_pedido_del`	`pedido`: BEFORE DELETE	Descuenta las líneas del pedido antes de que el borrado en cascada se las lleve.
`trg_ventas_dia_pedido_upd`	`pedido`: UPDATE OF fecha, estado	Saca el pedido de donde estaba y lo pone donde corresponde; si se cancela, solo lo saca.
`trg_ventas_dia_producto`	`producto`: UPDATE OF id_categoria	Pasa las ventas del producto de una categoría a la otra.

- La tabla tiene PK (`dia`, `id_categoria`), que es el índice que usa el panel, y `CHECK` de unidades positivas y monto no negativo.
- Las sumas usan `INSERT … ON CONFLICT`. Las restas, no: PostgreSQL controla el `CHECK` sobre la fila propuesta antes de detectar el conflicto. Por eso la resta es un `UPDATE`, y la fila se borra si queda en 0 unidades.
- Las funciones son `SECURITY DEFINER` con `search_path` fijo: el usuario de la aplicación no necesita permisos sobre la tabla resumen.
- La carga inicial (3 654 filas) se hace en la misma transacción que crea los disparadores, con las tablas bloqueadas contra escrituras: ninguna venta entra entre la carga y la activación.

d) El mismo reporte desde la estructura desnormalizada

```
SELECT c.nombre AS categoria, v.total_vendido
  FROM ventas_dia_categoria v
  JOIN categoria c ON c.id_categoria = v.id_categoria
 WHERE v.dia = hoy()
 ORDER BY v.total_vendido DESC
 LIMIT 5;
```

	Antes (4 tablas)	Después (resumen)
Tiempo de ejecución	**164,528 ms**	**0,086 ms**
Nodo dominante	`Seq Scan` on `pedido`: lee 200 005 filas y descarta 199 790	`Bitmap Index Scan` sobre `ventas_dia_categoria_pkey` y `Bitmap Heap Scan`: lee 5 filas
Bloques leídos (buffers)	5 313	8

#	Categoría	Total del día
1	Almacén	$ 131671,29
2	Congelados	$ 127249,80
3	Panificados	$ 97053,11
4	Lácteos	$ 93984,01
5	Bebidas	$ 90490,17

Mismo resultado por los dos caminos: 0 diferencias con `EXCEPT` en los dos sentidos.

La consulta es unas 1 913 veces más rápida y su costo ya no depende de cuántos pedidos haya: lee una fila por categoría.

e) Auditoría

```
CREATE VIEW v_auditoria_ventas_dia AS
WITH real AS (
    SELECT dia_venta(ped.fecha) AS dia, pr.id_categoria,
           SUM(dp.cantidad) AS unidades, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
      FROM detalle_pedido dp
      JOIN producto pr  ON pr.id_producto = dp.id_producto
      JOIN pedido   ped ON ped.id_pedido  = dp.id_pedido
     WHERE ped.estado <> 'CANCELADO'
     GROUP BY 1, 2)
SELECT COALESCE(r.dia, v.dia) AS dia, COALESCE(r.id_categoria, v.id_categoria) AS id_categoria,
       r.unidades AS unidades_reales, v.unidades AS unidades_resumen,
       r.total_vendido AS total_real, v.total_vendido AS total_resumen
  FROM real r FULL JOIN ventas_dia_categoria v ON v.dia = r.dia AND v.id_categoria = r.id_categoria
 WHERE r.unidades IS DISTINCT FROM v.unidades OR r.total_vendido IS DISTINCT FROM v.total_vendido;

SELECT * FROM v_auditoria_ventas_dia;   -- sobre la base ya migrada: (0 filas)
```

Recalcula el resumen desde la fuente de verdad y lo compara con la tabla. El `FULL JOIN` detecta montos distintos, filas que faltan y filas que sobran. Sobre la base migrada devolvió **(0 filas)**. También la corrí después de cada prueba de sincronización, todas dentro de `BEGIN … ROLLBACK`: venta nueva con dos líneas, cambio de cantidad, cancelación, borrado del pedido con sus líneas y cambio de categoría de un producto. Dio **0 en las cinco**.

Planes de EXPLAIN ANALYZE (capturas de mi corrida)

Antes: consulta original sobre las cuatro tablas (en rojo, el nodo dominante)

```
 Limit  (cost=16833.41..16833.43 rows=5 width=41) (actual time=164.487..164.490 rows=5.00 loops=1)
   Buffers: shared hit=5313
   ->  Sort  (cost=16833.41..16833.43 rows=5 width=41) (actual time=164.486..164.488 rows=5.00 loops=1)
         Sort Key: (sum(((dp.cantidad)::numeric * dp.precio_unitario))) DESC
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=5313
         ->  GroupAggregate  (cost=0.84..16833.36 rows=5 width=41) (actual time=164.229..164.478 rows=5.00 loops=1)
               Group Key: c.nombre
               Buffers: shared hit=5313
               ->  Nested Loop  (cost=0.84..16815.35 rows=1794 width=19) (actual time=2.318..164.338 rows=437.00 loops=1)
                     Join Filter: (c.id_categoria = pr.id_categoria)
                     Rows Removed by Join Filter: 1748
                     Buffers: shared hit=5313
                     ->  Index Scan using categoria_nombre_key on categoria c  (cost=0.13..12.21 rows=5 width=17) (actual
time=0.006..0.008 rows=5.00 loops=1)
                           Index Searches: 1
                           Buffers: shared hit=2
                     ->  Materialize  (cost=0.71..16673.08 rows=1794 width=18) (actual time=0.399..32.828 rows=437.00 loops=5)
                           Storage: Memory  Maximum Storage: 37kB
                           Buffers: shared hit=5311
                           ->  Nested Loop  (cost=0.71..16664.11 rows=1794 width=18) (actual time=1.993..163.921 rows=437.00 loops=1)
                                 Buffers: shared hit=5311
                                 ->  Nested Loop  (cost=0.42..16100.70 rows=1794 width=18) (actual time=1.984..162.443 rows=437.00
loops=1)
                                       Buffers: shared hit=4000
                                       ->  Seq Scan on pedido ped  (cost=0.00..8638.14 rows=897 width=8) (actual time=1.971..161.429
rows=215.00 loops=1)
                                             Filter: ((estado <> 'CANCELADO'::estado_pedido_enum) AND (((fecha AT TIME ZONE 'America/
Argentina/Buenos_Aires'::text))::date = ((now() AT TIME ZONE 'America/Argentina/Buenos_Aires'::text))::date))
                                             Rows Removed by Filter: 199790
                                             Buffers: shared hit=3138
                                       ->  Index Scan using detalle_pedido_id_pedido_id_producto_key on detalle_pedido dp
(cost=0.42..8.30 rows=2 width=26) (actual time=0.004..0.004 rows=2.03 loops=215)
                                             Index Cond: (id_pedido = ped.id_pedido)
                                             Index Searches: 215
```

```
                                        Buffers: shared hit=862
                                  ->  Index Scan using producto_pkey on producto pr  (cost=0.29..0.31 rows=1 width=16) (actual
time=0.003..0.003 rows=1.00 loops=437)
                                        Index Cond: (id_producto = dp.id_producto)
                                        Index Searches: 437
                                        Buffers: shared hit=1311
 Planning:
   Buffers: shared hit=35
 Planning Time: 0.759 ms
 Execution Time: 164.528 ms
```

Después: consulta sobre `ventas_dia_categoria`

```
 Limit  (cost=34.99..35.00 rows=5 width=27) (actual time=0.059..0.060 rows=5.00 loops=1)
   Buffers: shared hit=8
   ->  Sort  (cost=34.99..35.04 rows=19 width=27) (actual time=0.058..0.059 rows=5.00 loops=1)
         Sort Key: v.total_vendido DESC
         Sort Method: quicksort  Memory: 25kB
         Buffers: shared hit=8
         ->  Hash Join  (cost=5.55..34.67 rows=19 width=27) (actual time=0.051..0.055 rows=5.00 loops=1)
               Hash Cond: (v.id_categoria = c.id_categoria)
               Buffers: shared hit=8
               ->  Bitmap Heap Scan on ventas_dia_categoria v  (cost=4.43..33.51 rows=19 width=26) (actual time=0.027..0.029
rows=5.00 loops=1)
                     Recheck Cond: (dia = ((now() AT TIME ZONE 'America/Argentina/Buenos_Aires'::text))::date)
                     Heap Blocks: exact=5
                     Buffers: shared hit=7
                     ->  Bitmap Index Scan on ventas_dia_categoria_pkey  (cost=0.00..4.43 rows=19 width=0) (actual time=0.016..0.017
rows=5.00 loops=1)
                           Index Cond: (dia = ((now() AT TIME ZONE 'America/Argentina/Buenos_Aires'::text))::date)
                           Index Searches: 1
                           Buffers: shared hit=2
               ->  Hash  (cost=1.05..1.05 rows=5 width=17) (actual time=0.019..0.019 rows=5.00 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     Buffers: shared hit=1
                     ->  Seq Scan on categoria c  (cost=0.00..1.05 rows=5 width=17) (actual time=0.015..0.016 rows=5.00 loops=1)
                           Buffers: shared hit=1
 Planning Time: 0.146 ms
 Execution Time: 0.086 ms
```

Declaración de uso de IA (DUIA)

Herramienta	Claude (Anthropic), desde claude.ai.
Prompt utilizado	La guía del TP en PDF y el esquema de Food Store del repositorio.
Qué generó	Los dos scripts SQL, `ejecutar_tp_u4.sh`, el README de la carpeta y el borrador de este informe, escrito con los números de mis evidencias.
Qué se aceptó	Los scripts, después de probarlos en un entorno con PostgreSQL 16 y de correrlos en mi PC.
Qué se modificó o descartó	La consulta de la consigna se adaptó al esquema real. Se descartó `INSERT … ON CONFLICT` para las restas, por el `CHECK`. En la primera corrida en mi PC las categorías se cargaron con la codificación de la consola de Windows («AlmacÃ©n»): se fijó UTF-8 en el script y se volvió a correr.
Verificación realizada	Corrí `ejecutar_tp_u4.sh` en mi PC y controlé el resumen: en la Parte 1, los dos errores esperados y 0 tuplas perdidas y 0 espurias; en la Parte 2, 0 errores, auditoría vacía, el mismo top 5 por los dos caminos y la auditoría en 0 después de cada prueba. Nunca le pasé a la IA datos personales ni contraseñas.

Anexo A · `sql/tp_fnbc_control_lote.sql`

```
-- TP Unidad 4 · Parte 1 — FNBC sobre control_lote_almacen
-- Esquema original, instancia de ejemplo, validación de las DF, anomalías, tablas descompuestas,
-- vista de compatibilidad y migración verificada.
--
-- Se corre como postgres sobre una COPIA de la base Food Store (foodstore_tp4, ver ejecutar_tp_u4.sh).
-- Todo lo de la extensión mayorista va en el esquema tp4, para no mezclarse con el resto del proyecto.
-- Se puede volver a correr: arranca borrando lo que creó antes.
-- Abreviaturas: L = LoteID (lote_id), D = DepositoID (deposito_id), R = ResponsableControlID
-- (responsable_control_id).
\set ON_ERROR_STOP on

DROP SCHEMA IF EXISTS tp4 CASCADE;
CREATE SCHEMA tp4;
SET search_path TO tp4, public;

-- 1. Maestras de la extensión mayorista ------------------------------------------------
-- La consigna las da por existentes. En la copia no están, así que se crean con lo mínimo.
CREATE TABLE lote     (id BIGINT PRIMARY KEY, descripcion TEXT NOT NULL);
CREATE TABLE deposito (id BIGINT PRIMARY KEY, nombre      TEXT NOT NULL);
INSERT INTO lote     VALUES (501, 'Lote 501'), (502, 'Lote 502'), (503, 'Lote 503');
INSERT INTO deposito VALUES (30, 'Depósito 30'), (31, 'Depósito 31'), (32, 'Depósito 32');

CREATE EXTENSION IF NOT EXISTS pgcrypto WITH SCHEMA public;   -- ya existe desde la U5; por las dudas

-- Los responsables son usuarios de Food Store (REFERENCES usuario(id)). Se crean 801, 802 y 803
-- como usuarios de prueba, con datos inventados y una contraseña al azar que nadie conoce.
INSERT INTO public.usuario (id, nombre, apellido, mail, contrasena, rol)
OVERRIDING SYSTEM VALUE
SELECT r, 'Responsable', r::TEXT, 'responsable' || r || '@foodstore.test',
       crypt(gen_random_uuid()::TEXT, gen_salt('bf')), 'USUARIO'
  FROM unnest(ARRAY[801, 802, 803]) AS r
ON CONFLICT (id) DO NOTHING;

-- 2. Esquema original e instancia de ejemplo (tal como la consigna) -------------------------
CREATE TABLE control_lote_almacen (
    lote_id                BIGINT NOT NULL REFERENCES lote(id),
    deposito_id            BIGINT NOT NULL REFERENCES deposito(id),
    responsable_control_id BIGINT NOT NULL REFERENCES usuario(id),
    PRIMARY KEY (lote_id, deposito_id)
);
INSERT INTO control_lote_almacen VALUES
    (501, 30, 801),
    (502, 30, 801),
    (503, 31, 802);

\echo '== Instancia de ejemplo'
SELECT * FROM control_lote_almacen ORDER BY lote_id;

-- 3. a) Las DF, comprobadas sobre la instancia -----------------------------------------------
\echo '== DF1: {LoteID, DepositoID} -> ResponsableControlID (se esperan 0 violaciones)'
SELECT COUNT(*) AS violaciones_df1 FROM (
    SELECT lote_id, deposito_id FROM control_lote_almacen
     GROUP BY 1, 2 HAVING COUNT(DISTINCT responsable_control_id) > 1) x;
\echo '== DF2: ResponsableControlID -> DepositoID (se esperan 0 violaciones)'
SELECT COUNT(*) AS violaciones_df2 FROM (
    SELECT responsable_control_id FROM control_lote_almacen
     GROUP BY 1 HAVING COUNT(DISTINCT deposito_id) > 1) x;
\echo '== b) {ResponsableControlID} no es superclave: el responsable 801 aparece en dos filas'
SELECT responsable_control_id, COUNT(*) AS filas, array_agg(lote_id ORDER BY lote_id) AS lotes
  FROM control_lote_almacen GROUP BY 1 ORDER BY 1;
\echo '== b) {LoteID, ResponsableControlID} identifica cada fila (clave candidata, se espera 0)'
SELECT COUNT(*) AS repetidos FROM (
    SELECT lote_id, responsable_control_id FROM control_lote_almacen
     GROUP BY 1, 2 HAVING COUNT(*) > 1) x;

-- 4. d) Las tres anomalías, cada una dentro de BEGIN ... ROLLBACK ----------------------------
\echo '== d) Anomalía de actualización: 801 pasa al depósito 32, pero se cambia una sola de sus filas'
BEGIN;
UPDATE control_lote_almacen SET deposito_id = 32 WHERE lote_id = 501 AND responsable_control_id = 801;
SELECT responsable_control_id, array_agg(DISTINCT deposito_id) AS depositos
  FROM control_lote_almacen WHERE responsable_control_id = 801 GROUP BY 1;
ROLLBACK;

\echo '== d) Anomalía de inserción: el responsable 803 pertenece al depósito 31 pero no controló ningún lote'
\set ON_ERROR_STOP off
BEGIN;
INSERT INTO control_lote_almacen VALUES (NULL, 31, 803);
ROLLBACK;
\set ON_ERROR_STOP on

\echo '== d) Anomalía de borrado: se anula el control del lote 503 y se pierde que 802 es del depósito 31'
BEGIN;
DELETE FROM control_lote_almacen WHERE lote_id = 503;
SELECT COUNT(*) AS filas_que_dicen_el_deposito_de_802
  FROM control_lote_almacen WHERE responsable_control_id = 802;
ROLLBACK;

-- 5. e) Descomposición por la DF violatoria R -> D -------------------------------------------
-- R1(R, D) con clave R conserva R -> D. R2(L, R) con clave {L, R}. El atributo común es R.
CREATE TABLE responsable_deposito (
```

```
    responsable_control_id BIGINT PRIMARY KEY REFERENCES usuario(id),
    deposito_id            BIGINT NOT NULL    REFERENCES deposito(id)
);
CREATE TABLE control_lote (
    lote_id                BIGINT NOT NULL REFERENCES lote(id),
    responsable_control_id BIGINT NOT NULL REFERENCES responsable_deposito(responsable_control_id),
    PRIMARY KEY (lote_id, responsable_control_id)
);
-- Vista de compatibilidad: reconstruye la relación original con una reunión natural (por R).
CREATE VIEW v_control_lote_almacen AS
SELECT lote_id, deposito_id, responsable_control_id
  FROM control_lote NATURAL JOIN responsable_deposito;

-- La descomposición no preserva DF1 ({L, D} -> R): ninguna tabla tiene L y D juntos. Sin una
-- salvaguarda, dos responsables del mismo depósito podrían controlar el mismo lote. El trigger
-- busca en la reunión algún (L, D) repetido y rechaza la operación. El candado de transacción
-- evita que dos transacciones concurrentes pasen el control al mismo tiempo.
CREATE FUNCTION fn_control_lote_df1() RETURNS TRIGGER
LANGUAGE plpgsql SET search_path = tp4, public, pg_temp AS $$
DECLARE v_lote BIGINT; v_dep BIGINT;
BEGIN
    PERFORM pg_advisory_xact_lock(hashtext('tp4.control_lote_df1'));
    SELECT lote_id, deposito_id INTO v_lote, v_dep
      FROM v_control_lote_almacen GROUP BY 1, 2 HAVING COUNT(*) > 1 LIMIT 1;
    IF FOUND THEN
        RAISE EXCEPTION 'El lote % ya tiene un responsable del depósito % (DF1: {lote, depósito} -> responsable)',
                        v_lote, v_dep;
    END IF;
    RETURN NULL;
END $$;
CREATE TRIGGER trg_control_lote_df1 AFTER INSERT OR UPDATE ON control_lote
    FOR EACH STATEMENT EXECUTE FUNCTION fn_control_lote_df1();
CREATE TRIGGER trg_responsable_deposito_df1 AFTER UPDATE OF deposito_id ON responsable_deposito
    FOR EACH STATEMENT EXECUTE FUNCTION fn_control_lote_df1();

-- 6. f) Migración de la instancia, en una sola transacción y verificada ----------------------
BEGIN;
INSERT INTO responsable_deposito (responsable_control_id, deposito_id)
SELECT DISTINCT responsable_control_id, deposito_id FROM control_lote_almacen;
INSERT INTO control_lote (lote_id, responsable_control_id)
SELECT lote_id, responsable_control_id FROM control_lote_almacen;
-- Si la reunión no da exactamente la relación original, se cancela todo.
DO $$
DECLARE v_perdidas INT; v_espurias INT;
BEGIN
    SELECT COUNT(*) INTO v_perdidas FROM (SELECT * FROM control_lote_almacen
                                          EXCEPT SELECT * FROM v_control_lote_almacen) x;
    SELECT COUNT(*) INTO v_espurias FROM (SELECT * FROM v_control_lote_almacen
                                          EXCEPT SELECT * FROM control_lote_almacen) x;
    IF v_perdidas + v_espurias > 0 THEN
        RAISE EXCEPTION 'La reunión no es equivalente: % perdidas, % espurias', v_perdidas, v_espurias;
    END IF;
END $$;
COMMIT;

\echo '== f) Verificación de la equivalencia con la vista de compatibilidad (se espera 0 y 0)'
SELECT (SELECT COUNT(*) FROM (SELECT * FROM control_lote_almacen
                              EXCEPT SELECT * FROM v_control_lote_almacen) x) AS tuplas_perdidas,
       (SELECT COUNT(*) FROM (SELECT * FROM v_control_lote_almacen
                              EXCEPT SELECT * FROM control_lote_almacen) x) AS tuplas_espurias;
\echo '== Tablas descompuestas'
SELECT * FROM responsable_deposito ORDER BY 1;
SELECT * FROM control_lote ORDER BY 1, 2;
\echo '== Reunión desde la vista de compatibilidad'
SELECT * FROM v_control_lote_almacen ORDER BY lote_id;

-- 7. Las anomalías sobre el esquema nuevo (dentro de BEGIN ... ROLLBACK) --------------------
\echo '== Inserción: 803 se registra en el depósito 31 sin haber controlado lotes'
\echo '== Actualización: el depósito de 801 cambia en una sola fila (UPDATE 1)'
\echo '== Borrado: se anula el control del lote 503 y el depósito de 802 sigue registrado'
BEGIN;
INSERT INTO responsable_deposito VALUES (803, 31);
UPDATE responsable_deposito SET deposito_id = 32 WHERE responsable_control_id = 801;
DELETE FROM control_lote WHERE lote_id = 503;
SELECT * FROM responsable_deposito ORDER BY 1;
SELECT * FROM v_control_lote_almacen ORDER BY lote_id;
ROLLBACK;

\echo '== DF1 en el esquema nuevo: el lote 501 se fracciona y lo controla también 802, del depósito 31 (se acepta)'
BEGIN;
INSERT INTO control_lote VALUES (501, 802);
SELECT * FROM v_control_lote_almacen WHERE lote_id = 501 ORDER BY deposito_id;
ROLLBACK;
\echo '== DF1 en el esquema nuevo: un segundo responsable del depósito 31 para el lote 503 (se espera ERROR)'
\set ON_ERROR_STOP off
BEGIN;
INSERT INTO responsable_deposito VALUES (803, 31);
INSERT INTO control_lote VALUES (503, 803);
ROLLBACK;
\set ON_ERROR_STOP on
```

Anexo B · `sql/tp_desnormalizacion_top_categorias.sql`

```
-- TP Unidad 4 · Parte 2 — Top 5 de categorías del día con desnormalización controlada
-- Estructura desnormalizada (tp4.ventas_dia_categoria), su sincronización con disparadores,
-- la consulta del punto 5.2(d) y el script de auditoría del punto 5.2(e).
--
-- Se corre como postgres sobre la copia foodstore_tp4 (ver ejecutar_tp_u4.sh).
-- Adaptación al esquema real de Food Store (la consigna usa nombres genéricos):
--   * no hay dp.subtotal: el monto de una línea es cantidad * precio_unitario;
--   * las claves se llaman id_producto, id_categoria e id_pedido;
--   * pedido y detalle_pedido no tienen borrado lógico. Lo que no cuenta como venta es un
--     pedido CANCELADO (columna estado, de la U7). Un pedido borrado se borra con sus líneas;
--   * pedido.fecha es TIMESTAMPTZ: «el día» es la fecha en la hora de Argentina.
\set ON_ERROR_STOP on
CREATE SCHEMA IF NOT EXISTS tp4;
SET search_path TO tp4, public;

-- Limpieza, para poder volver a correrlo ------------------------------------------------
DROP TRIGGER IF EXISTS trg_ventas_dia_detalle   ON public.detalle_pedido;
DROP TRIGGER IF EXISTS trg_ventas_dia_pedido_del ON public.pedido;
DROP TRIGGER IF EXISTS trg_ventas_dia_pedido_upd ON public.pedido;
DROP TRIGGER IF EXISTS trg_ventas_dia_producto  ON public.producto;
DROP VIEW  IF EXISTS v_auditoria_ventas_dia;
DROP TABLE IF EXISTS ventas_dia_categoria;

-- El día de una venta, siempre en la hora de Argentina (no depende de la zona de la sesión)
CREATE OR REPLACE FUNCTION dia_venta(p TIMESTAMPTZ) RETURNS DATE
LANGUAGE sql IMMUTABLE AS $$ SELECT (p AT TIME ZONE 'America/Argentina/Buenos_Aires')::DATE $$;
CREATE OR REPLACE FUNCTION hoy() RETURNS DATE
LANGUAGE sql STABLE AS $$ SELECT (now() AT TIME ZONE 'America/Argentina/Buenos_Aires')::DATE $$;

-- ============================================================================
-- 5.2(a) La consulta original, adaptada al esquema, con EXPLAIN ANALYZE
-- ============================================================================
ANALYZE public.pedido, public.detalle_pedido, public.producto, public.categoria;
\echo '== Tamaño de la base y ventas de hoy (hora de Argentina)'
SELECT (SELECT COUNT(*) FROM pedido) AS pedidos, (SELECT COUNT(*) FROM detalle_pedido) AS lineas,
       (SELECT COUNT(*) FROM pedido WHERE dia_venta(fecha) = hoy() AND estado <> 'CANCELADO') AS pedidos_de_hoy,
       hoy() AS hoy;

\echo '== 5.2(a) Consulta original: recorre y une detalle_pedido, producto, categoria y pedido'
SELECT c.nombre AS categoria, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
  FROM detalle_pedido dp
  JOIN producto  pr  ON pr.id_producto   = dp.id_producto
  JOIN categoria c   ON c.id_categoria   = pr.id_categoria
  JOIN pedido    ped ON ped.id_pedido    = dp.id_pedido
 WHERE dia_venta(ped.fecha) = hoy()
   AND ped.estado <> 'CANCELADO'
 GROUP BY c.nombre
 ORDER BY total_vendido DESC
 LIMIT 5;
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre AS categoria, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
  FROM detalle_pedido dp
  JOIN producto  pr  ON pr.id_producto   = dp.id_producto
  JOIN categoria c   ON c.id_categoria   = pr.id_categoria
  JOIN pedido    ped ON ped.id_pedido    = dp.id_pedido
 WHERE dia_venta(ped.fecha) = hoy()
   AND ped.estado <> 'CANCELADO'
 GROUP BY c.nombre
 ORDER BY total_vendido DESC
 LIMIT 5;

-- ============================================================================
-- 5.2(c) Estructura desnormalizada y su mecanismo de sincronización
-- ============================================================================
BEGIN;
-- Una fila por día y categoría con lo vendido. Es un dato derivado: la fuente de verdad
-- siguen siendo pedido y detalle_pedido, y la tabla se puede reconstruir en cualquier momento.
CREATE TABLE ventas_dia_categoria (
    dia            DATE          NOT NULL,
    id_categoria   BIGINT        NOT NULL REFERENCES public.categoria (id_categoria),
    unidades       BIGINT        NOT NULL CHECK (unidades > 0),
    total_vendido  NUMERIC(14,2) NOT NULL CHECK (total_vendido >= 0),
    PRIMARY KEY (dia, id_categoria)          -- el panel busca por día: WHERE dia = hoy()
);

-- Suma (o resta) una variación en la fila de ese día y categoría. Con una variación positiva
-- alcanza un INSERT ... ON CONFLICT. Con una negativa no: PostgreSQL controla el CHECK sobre la
-- fila propuesta antes de detectar el conflicto (el mismo error de la U4 del TPI). Por eso la
-- resta es un UPDATE, y si la fila queda en 0 unidades se borra.
CREATE OR REPLACE FUNCTION fn_ventas_ajustar(p_dia DATE, p_categoria BIGINT, p_unidades BIGINT, p_monto NUMERIC)
RETURNS VOID LANGUAGE plpgsql SECURITY DEFINER SET search_path = tp4, public, pg_temp AS $$
BEGIN
    IF p_unidades = 0 AND p_monto = 0 THEN RETURN; END IF;
    IF p_unidades > 0 THEN
        INSERT INTO ventas_dia_categoria AS v (dia, id_categoria, unidades, total_vendido)
        VALUES (p_dia, p_categoria, p_unidades, p_monto)
        ON CONFLICT (dia, id_categoria) DO UPDATE
           SET unidades = v.unidades + EXCLUDED.unidades,
               total_vendido = v.total_vendido + EXCLUDED.total_vendido;
    ELSE
```

```
        UPDATE ventas_dia_categoria
           SET unidades = unidades + p_unidades, total_vendido = total_vendido + p_monto
         WHERE dia = p_dia AND id_categoria = p_categoria;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'ventas_dia_categoria desincronizada: no hay fila para % / categoría %', p_dia, p_categoria;
        END IF;
        DELETE FROM ventas_dia_categoria
         WHERE dia = p_dia AND id_categoria = p_categoria AND unidades = 0;
    END IF;
END $$;

-- Línea de pedido nueva, cambiada o borrada. Si el pedido ya no existe es porque se está
-- borrando el pedido entero: eso lo descontó antes trg_ventas_dia_pedido_del.
CREATE OR REPLACE FUNCTION fn_ventas_dia_detalle() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = tp4, public, pg_temp AS $$
DECLARE v_fecha TIMESTAMPTZ; v_estado TEXT; v_cat BIGINT;
BEGIN
    IF TG_OP IN ('UPDATE', 'DELETE') THEN
        SELECT fecha, estado INTO v_fecha, v_estado FROM pedido WHERE id_pedido = OLD.id_pedido;
        IF FOUND AND v_estado <> 'CANCELADO' THEN
            SELECT id_categoria INTO v_cat FROM producto WHERE id_producto = OLD.id_producto;
            PERFORM fn_ventas_ajustar(dia_venta(v_fecha), v_cat, -OLD.cantidad, -(OLD.cantidad * OLD.precio_unitario));
        END IF;
    END IF;
    IF TG_OP IN ('INSERT', 'UPDATE') THEN
        SELECT fecha, estado INTO v_fecha, v_estado FROM pedido WHERE id_pedido = NEW.id_pedido;
        IF FOUND AND v_estado <> 'CANCELADO' THEN
            SELECT id_categoria INTO v_cat FROM producto WHERE id_producto = NEW.id_producto;
            PERFORM fn_ventas_ajustar(dia_venta(v_fecha), v_cat, NEW.cantidad, NEW.cantidad * NEW.precio_unitario);
        END IF;
    END IF;
    RETURN NULL;
END $$;

-- Suma o resta todas las líneas de un pedido (signo +1 o -1), agrupadas por categoría
CREATE OR REPLACE FUNCTION fn_ventas_pedido(p_pedido BIGINT, p_fecha TIMESTAMPTZ, p_signo INT)
RETURNS VOID LANGUAGE plpgsql SECURITY DEFINER SET search_path = tp4, public, pg_temp AS $$
DECLARE r RECORD;
BEGIN
    FOR r IN SELECT pr.id_categoria, SUM(dp.cantidad) AS u, SUM(dp.cantidad * dp.precio_unitario) AS m
               FROM detalle_pedido dp JOIN producto pr ON pr.id_producto = dp.id_producto
              WHERE dp.id_pedido = p_pedido GROUP BY 1 LOOP
        PERFORM fn_ventas_ajustar(dia_venta(p_fecha), r.id_categoria, p_signo * r.u, p_signo * r.m);
    END LOOP;
END $$;

-- Pedido borrado: se descuentan sus líneas ANTES de que el borrado en cascada se las lleve
CREATE OR REPLACE FUNCTION fn_ventas_dia_pedido_del() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = tp4, public, pg_temp AS $$
BEGIN
    IF OLD.estado <> 'CANCELADO' THEN PERFORM fn_ventas_pedido(OLD.id_pedido, OLD.fecha, -1); END IF;
    RETURN OLD;
END $$;

-- Pedido que cambia de fecha o de estado (por ejemplo, se cancela): sale de donde estaba
-- y entra donde corresponde
CREATE OR REPLACE FUNCTION fn_ventas_dia_pedido_upd() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = tp4, public, pg_temp AS $$
BEGIN
    IF OLD.estado <> 'CANCELADO' THEN PERFORM fn_ventas_pedido(OLD.id_pedido, OLD.fecha, -1); END IF;
    IF NEW.estado <> 'CANCELADO' THEN PERFORM fn_ventas_pedido(NEW.id_pedido, NEW.fecha, +1); END IF;
    RETURN NULL;
END $$;

-- Producto que cambia de categoría: sus ventas pasan de una categoría a la otra
CREATE OR REPLACE FUNCTION fn_ventas_dia_producto() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = tp4, public, pg_temp AS $$
DECLARE r RECORD;
BEGIN
    FOR r IN SELECT dia_venta(ped.fecha) AS dia, SUM(dp.cantidad) AS u, SUM(dp.cantidad * dp.precio_unitario) AS m
               FROM detalle_pedido dp JOIN pedido ped ON ped.id_pedido = dp.id_pedido
              WHERE dp.id_producto = NEW.id_producto AND ped.estado <> 'CANCELADO' GROUP BY 1 LOOP
        PERFORM fn_ventas_ajustar(r.dia, OLD.id_categoria, -r.u, -r.m);
        PERFORM fn_ventas_ajustar(r.dia, NEW.id_categoria,  r.u,  r.m);
    END LOOP;
    RETURN NULL;
END $$;

CREATE TRIGGER trg_ventas_dia_detalle AFTER INSERT OR UPDATE OR DELETE ON public.detalle_pedido
    FOR EACH ROW EXECUTE FUNCTION fn_ventas_dia_detalle();
CREATE TRIGGER trg_ventas_dia_pedido_del BEFORE DELETE ON public.pedido
    FOR EACH ROW EXECUTE FUNCTION fn_ventas_dia_pedido_del();
CREATE TRIGGER trg_ventas_dia_pedido_upd AFTER UPDATE OF fecha, estado ON public.pedido
    FOR EACH ROW WHEN (OLD.fecha IS DISTINCT FROM NEW.fecha OR OLD.estado IS DISTINCT FROM NEW.estado)
    EXECUTE FUNCTION fn_ventas_dia_pedido_upd();
CREATE TRIGGER trg_ventas_dia_producto AFTER UPDATE OF id_categoria ON public.producto
    FOR EACH ROW WHEN (OLD.id_categoria IS DISTINCT FROM NEW.id_categoria)
    EXECUTE FUNCTION fn_ventas_dia_producto();

-- Carga inicial desde la fuente de verdad. Las tablas quedan bloqueadas contra escrituras
-- hasta el COMMIT: ninguna venta puede entrar entre la carga y la activación de los triggers.
LOCK TABLE public.pedido, public.detalle_pedido, public.producto IN SHARE ROW EXCLUSIVE MODE;
INSERT INTO ventas_dia_categoria (dia, id_categoria, unidades, total_vendido)
SELECT dia_venta(ped.fecha), pr.id_categoria, SUM(dp.cantidad), SUM(dp.cantidad * dp.precio_unitario)
  FROM detalle_pedido dp
```

```
  JOIN producto pr  ON pr.id_producto = dp.id_producto
  JOIN pedido   ped ON ped.id_pedido  = dp.id_pedido
 WHERE ped.estado <> 'CANCELADO'
 GROUP BY 1, 2;
COMMIT;

\echo '== Filas de la estructura desnormalizada'
SELECT COUNT(*) AS filas, MIN(dia) AS primer_dia, MAX(dia) AS ultimo_dia FROM ventas_dia_categoria;

-- ============================================================================
-- 5.2(e) Auditoría: diferencias entre el dato redundante y su fuente de verdad
-- ============================================================================
CREATE VIEW v_auditoria_ventas_dia AS
WITH real AS (
    SELECT dia_venta(ped.fecha) AS dia, pr.id_categoria,
           SUM(dp.cantidad) AS unidades, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
      FROM detalle_pedido dp
      JOIN producto pr  ON pr.id_producto = dp.id_producto
      JOIN pedido   ped ON ped.id_pedido  = dp.id_pedido
     WHERE ped.estado <> 'CANCELADO'
     GROUP BY 1, 2)
SELECT COALESCE(r.dia, v.dia) AS dia, COALESCE(r.id_categoria, v.id_categoria) AS id_categoria,
       r.unidades AS unidades_reales, v.unidades AS unidades_resumen,
       r.total_vendido AS total_real, v.total_vendido AS total_resumen
  FROM real r FULL JOIN ventas_dia_categoria v ON v.dia = r.dia AND v.id_categoria = r.id_categoria
 WHERE r.unidades IS DISTINCT FROM v.unidades OR r.total_vendido IS DISTINCT FROM v.total_vendido;

\echo '== 5.2(e) Auditoría sobre la base migrada (se esperan 0 filas)'
SELECT * FROM v_auditoria_ventas_dia ORDER BY dia, id_categoria;

-- ============================================================================
-- 5.2(d) El mismo reporte, leyendo la estructura desnormalizada
-- ============================================================================
\echo '== 5.2(d) Top 5 del día desde ventas_dia_categoria (mismo resultado que 5.2(a))'
SELECT c.nombre AS categoria, v.total_vendido
  FROM ventas_dia_categoria v
  JOIN categoria c ON c.id_categoria = v.id_categoria
 WHERE v.dia = hoy()
 ORDER BY v.total_vendido DESC
 LIMIT 5;
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.nombre AS categoria, v.total_vendido
  FROM ventas_dia_categoria v
  JOIN categoria c ON c.id_categoria = v.id_categoria
 WHERE v.dia = hoy()
 ORDER BY v.total_vendido DESC
 LIMIT 5;

\echo '== Los dos caminos dan el mismo resultado (se esperan 0 diferencias)'
WITH antes AS (
    SELECT c.nombre AS categoria, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
      FROM detalle_pedido dp
      JOIN producto pr ON pr.id_producto = dp.id_producto
      JOIN categoria c ON c.id_categoria = pr.id_categoria
      JOIN pedido ped  ON ped.id_pedido  = dp.id_pedido
     WHERE dia_venta(ped.fecha) = hoy() AND ped.estado <> 'CANCELADO'
     GROUP BY c.nombre ORDER BY total_vendido DESC LIMIT 5),
despues AS (
    SELECT c.nombre AS categoria, v.total_vendido
      FROM ventas_dia_categoria v JOIN categoria c ON c.id_categoria = v.id_categoria
     WHERE v.dia = hoy() ORDER BY v.total_vendido DESC LIMIT 5)
SELECT (SELECT COUNT(*) FROM (SELECT * FROM antes EXCEPT SELECT * FROM despues) a)
     + (SELECT COUNT(*) FROM (SELECT * FROM despues EXCEPT SELECT * FROM antes) b) AS diferencias;

-- ============================================================================
-- Pruebas de la sincronización (todo dentro de BEGIN ... ROLLBACK)
-- ============================================================================
\echo '== Sincronización: venta nueva, cambio de cantidad, cancelación, borrado y cambio de categoría'
BEGIN;
CREATE TEMP TABLE prueba AS
SELECT (SELECT MIN(id_cliente) FROM cliente) AS cliente,
       (SELECT MIN(id_producto) FROM producto WHERE activo AND stock >= 10 AND id_categoria = 1) AS prod_a,
       (SELECT MIN(id_producto) FROM producto WHERE activo AND stock >= 10 AND id_categoria = 2) AS prod_b;
SELECT dia, id_categoria, unidades, total_vendido FROM ventas_dia_categoria
 WHERE dia = hoy() AND id_categoria IN (1, 2) ORDER BY 2;

\echo '-- 1. Venta nueva de hoy con dos líneas'
INSERT INTO pedido (forma_pago, id_cliente) SELECT 'EFECTIVO', cliente FROM prueba RETURNING id_pedido AS pedido_prueba \gset
INSERT INTO detalle_pedido (cantidad, precio_unitario, id_pedido, id_producto)
SELECT 2, 1000.00, :pedido_prueba, prod_a FROM prueba UNION ALL
SELECT 3,  500.00, :pedido_prueba, prod_b FROM prueba;
SELECT dia, id_categoria, unidades, total_vendido FROM ventas_dia_categoria
 WHERE dia = hoy() AND id_categoria IN (1, 2) ORDER BY 2;
SELECT COUNT(*) AS auditoria_tras_la_venta FROM v_auditoria_ventas_dia;

\echo '-- 2. Cambio de cantidad en una línea'
UPDATE detalle_pedido SET cantidad = 5 WHERE id_pedido = :pedido_prueba AND id_producto = (SELECT prod_a FROM prueba);
SELECT COUNT(*) AS auditoria_tras_el_cambio FROM v_auditoria_ventas_dia;

\echo '-- 3. El pedido se cancela'
UPDATE pedido SET estado = 'CANCELADO' WHERE id_pedido = :pedido_prueba;
SELECT COUNT(*) AS auditoria_tras_la_cancelacion FROM v_auditoria_ventas_dia;

\echo '-- 4. El pedido vuelve a PENDIENTE y después se borra con sus líneas'
UPDATE pedido SET estado = 'PENDIENTE' WHERE id_pedido = :pedido_prueba;
```

```
DELETE FROM pedido WHERE id_pedido = :pedido_prueba;
SELECT COUNT(*) AS auditoria_tras_el_borrado FROM v_auditoria_ventas_dia;

\echo '-- 5. Un producto cambia de categoría'
UPDATE producto SET id_categoria = 3 WHERE id_producto = (SELECT prod_a FROM prueba);
SELECT COUNT(*) AS auditoria_tras_cambiar_la_categoria FROM v_auditoria_ventas_dia;
ROLLBACK;

-- ============================================================================
-- Reversión: la desnormalización se quita sin perder información
-- ============================================================================
-- La fuente de verdad nunca se tocó, así que revertir es borrar los triggers, las funciones y
-- la tabla. Se ensaya dentro de una transacción y se deshace, para que el TP quede armado.
\echo '== Reversión (ensayo con ROLLBACK): se borra la desnormalización y la consulta original sigue igual'
BEGIN;
DROP TRIGGER trg_ventas_dia_detalle    ON public.detalle_pedido;
DROP TRIGGER trg_ventas_dia_pedido_del ON public.pedido;
DROP TRIGGER trg_ventas_dia_pedido_upd ON public.pedido;
DROP TRIGGER trg_ventas_dia_producto   ON public.producto;
DROP VIEW  v_auditoria_ventas_dia;
DROP TABLE ventas_dia_categoria;
SELECT c.nombre AS categoria, SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
  FROM detalle_pedido dp
  JOIN producto pr ON pr.id_producto = dp.id_producto
  JOIN categoria c ON c.id_categoria = pr.id_categoria
  JOIN pedido ped  ON ped.id_pedido  = dp.id_pedido
 WHERE dia_venta(ped.fecha) = hoy() AND ped.estado <> 'CANCELADO'
 GROUP BY c.nombre ORDER BY total_vendido DESC LIMIT 5;
ROLLBACK;
```